

Flow & IO

@author Yangxin Wu

Last edited: 20251129

- [流](#)
 - [流的分类](#)
- [流式输入输出](#)
 - [读取控制台输入](#)
 - [向控制台输出](#)
 - [流式文件 IO](#)
- [文件操作](#)
 - [路径类 Path](#)
 - [文件工具类 Files](#)
- [对象的序列化](#)

流

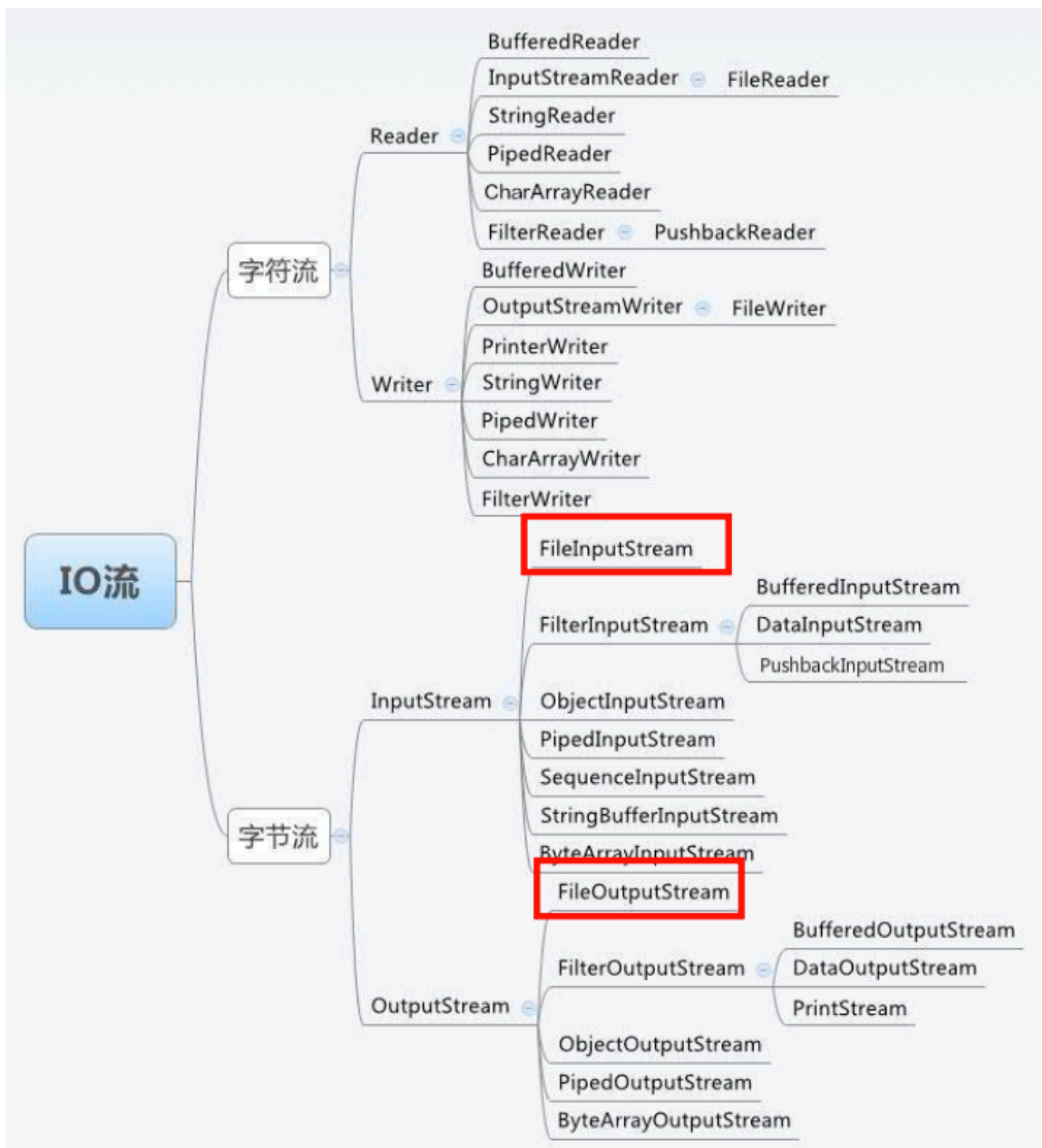
流是一组有序的数据序列，将数据从一个地方带到另一个地方。

在 Java 中，控制台的输入输出、文件的读写和网络通信都是通过「流」的方式进行的。

流的分类

- 按「方向」分：可以分为「输出流」和「输入流」两种。数据从外部进入程序为「输入」，反之为「输出」。
- 按「数据单位」分：可以分为「字节流」（传输字节，如文本、图像、声音等各种内容）和「字符流」（传输 16 bit 的字符，只能用来处理文本）。

Java 的所有的流位于 `java.io` 包中，类名以 `Stream` 结尾的流都是字节流，以 `Reader` 或 `Writer` 结尾的流都是字符流。



流式输入输出

每个Java程序都包含一个系统流位于 `java.lang.System`，包含三个子类：

- `System.in` 标准输入流，默认设备是键盘输入。相当于C的 `STDIN`。
- `System.out` 标准输出流，默认设备是控制台。相当于C的 `STDOUT`。
- `System.err` 标准错误流，默认设备是控制台。相当于C的 `STDERR`。

读取控制台输入

下面的程序示范了用 `read()` 方法从控制台不断读取字符直到用户输入 "q"。

```
import java.io.*;

public class BRRead {
    public static void main(String[] args) throws IOException {
        // 使用 System.in 创建 BufferedReader
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));

        System.out.println("输入字符，按下 q 键退出。");

        // 读取字符
        char c;
        do {
            c = (char) br.read();
            System.out.println(c);
        } while (c != 'q');
    }
}
```

下面的程序用 `readline()` 方法读取和显示字符行直到输入单词 "end"。

```
import java.io.*;

public class BRReadLines {
    public static void main(String[] args) throws IOException {
        // 使用 System.in 创建 BufferedReader
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));

        System.out.println("Enter lines of text.");
        System.out.println("Enter end to quit.");

        // 读取字符串
        String str;
        do {
            str = br.readLine();
            System.out.println(str);
        } while (!str.equals("end"))
    }
}
```

为了简化上面的过程，可以用 `Scanner` 代替手动创建 `BufferedReader`。参见 [Java 基础](#)。

向控制台输出

参见 [Java 基础](#)。

流式文件 IO

在 Java 中有两个重要的文件 IO 流 `FileInputStream` 和 `FileOutputStream`。前者用于从文件读取数据，后者用于向文件写入数据。

对 `FileInputStream` 和 `FileOutputStream` 可以分别使用 `InputStreamReader` 和 `OutputStreamWriter` 转化为字符流，从而实现字符读写。

代码实例如下：

```
import java.io.*;

public class FileStreamTest {
    public static void main(String[] args) throws IOException {
        File f = new File("a.txt");

        // 构建 FileOutputStream 对象,文件不存在会自动新建
        FileOutputStream fop = new FileOutputStream(f);

        // 构建 OutputStreamWriter 对象,参数可以指定编码,默认为操作系统默认编码
        OutputStreamWriter writer = new OutputStreamWriter(fop, "UTF-8");

        writer.append("中文输入");
        writer.append("\r\n");
        writer.append("English");

        // 关闭写入流,同时会把缓冲区内容写入文件
        writer.close();

        // 关闭输出流,释放系统资源
        fop.close();

        // 构建 FileInputStream 对象
        FileInputStream fip = new FileInputStream(f);

        // 构建 InputStreamReader 对象,编码与写入相同
        InputStreamReader reader = new InputStreamReader(fip, "UTF-8");

        StringBuffer sb = new StringBuffer();
        while (reader.ready()) {
            // 转成 char 加到 StringBuffer 对象中
            sb.append((char) reader.read());
        }
        System.out.println(sb.toString());

        // 关闭读取流
        reader.close();

        // 关闭输入流,释放系统资源
        fip.close();
    }
}
```

```
}  
}
```

文件操作

上面所说的基于流的文件操作是比较低级的、简单的操作。
在 `Java` 中，复杂的文件操作由 `Path` 和 `Files` 类封装。

路径类 `Path`

一个 `Path` 对象记录一个路径，可以是相对路径或绝对路径。
`Path` 对象使用 `Path.get` 工厂方法创建。

```
Path absolute = Paths.get("/home/hans/foobar.txt"); // 绝对路径  
Path relative = Paths.get("hello.txt"); // 相对路径
```

文件工具类 `Files`

`Files` 是一个操作文件的工具类，能对 `Path` 对象指向的文件进行各种各样的操作。

读文件：

- 逐字节读入文件的所有内容到 `byte` 序列：

```
byte[] bytes = Files.readAllBytes(path);
```

- 以文本方式读入文件（`charset` 是字符集，如 `StandardCharsets.UTF_8`）：

```
String content = Files.readString(path, charset);
```

- 以行读入文件：

```
List<String> lines = Files.readAllLines(path, charset);
```

写文件：

- 写一个字符串到文件中：

```
Files.writeString(path, content.charset());
```

- 向指定文件追加内容：

```
Files.write(path, content.getBytes(charset), StandardOpenOption.APPEND);
```

- 将一个行的集合写到文件中：

```
Files.write(path, lines, charset);
```

创建文件和目录：

- 创建新目录：

```
Files.createDirectory(path); // 路径中除最后一个部件外，其他部分都必须是已存在的。
```

- 创建空文件：

```
Files.createFile(path); // 如果文件已经存在了，那么这个调用就会抛出异常。
```

复制、移动和删除文件：

- 将文件从一个位置复制到另一个位置：

```
Files.copy(fromPath, toPath); // 如果目标路径已经存在，那么复制将失败。
```

```
// 如果想要覆盖已有的目标路径，可以使用 REPLACE_EXISTING 选项。
```

```
Files.copy(fromPath, toPath, standardCopyOption.REPLACE_EXISTING);
```

```
// 如果想要复制所有的文件属性，可以使用 COPY_ATTRIBUTES 选项。
```

```
Files.copy(fromPath, toPath, standardCopyOption.COPY_ATTRIBUTES);
```

- 移动文件（即复制并删除原文件）：

```
Files.move(fromPath, toPath); // 如果目标路径已经存在，那么移动将失败。
```

```
// 如果想要覆盖已有的目标路径，可以使用 REPLACE_EXISTING 选项。
```

```
Files.move(fromPath, toPath, standardCopyOption.REPLACE_EXISTING);
```

```
// 可以将移动操作定义为原子性的，这样可以保证要么移动操作成功，要么源文件继续保持在原来位置。
```

```
Files.move(fromPath, toPath, standardCopyOption.ATOMIC_MOVE);
```

- 删除文件：

```
Files.delete(path); // 如果要删除的文件不存在，这个方法就会抛出异常。
```

```
// 如果使用下面的方法，则可以移除空目录。
```

```
boolean deleted = Files.deleteIfExists(path);
```

获取文件信息：

下面的静态方法都将返回一个 `boolean` 值，表示检查路径的某个属性的结果：

- `exists()`
- `isHidden()`
- `isReadable()`, `isWritable()`, `isExecutable()`
- `isRegularFile()`, `isDirectory()`, `isSymbolicLink()`
- `Size()`

对象的序列化

可以通过让一个对象实现 `Serializable` 接口，使得它可以被序列化。

能够被序列化的对象可以被「打包」写进流中，也能从流中读出并变回对象。

首先有序列化的对象：

```
class Person implements Serializable {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // 为了方便查看对象，重写默认的 toString()
    @Override
    public String toString() {
        return "Person{" + "name='" + name + '\'' + ", age=" + age + "}";
    }
}
```

调用 `ObjectOutputStream` 对象的 `writeObject` 输出可序列化对象：

```
public class SeriDemo {
    public static void main(String[] args) {
        // ObjectOutputStream
        try {
            Person p1 = new Person("zhangsan", 30);
            System.out.println(p1);
            ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("person.dat")
            );
            oos.writeObject(p1);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
// ObjectInputStream
try (
    ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream("person.dat")
    )
){
    System.out .println("readObject(): ");
    Person zhangsan = (Person) ois.readObject();
    System.out .println(zhangsan);
} catch (Exception e) {
    e.printStackTrace();
}
}
```